

A COMPLETE PRACTICAL GUIDE

Designing Beautiful Web Pages with Bootstrap 5

From the 12-column grid to advanced Sass customization — the concepts, the math, the structure, and the design instincts behind professional, production-quality front-end interfaces.

Contents

PART I — FOUNDATIONS

01	Welcome to Bootstrap 5	4
02	Getting Bootstrap Into Your Project	7
03	How Bootstrap Thinks: Reboot, Box-Sizing & Breakpoints	10

PART II — THE GRID SYSTEM

04	The 12-Column Grid: Concept & Math	13
05	Containers Explained	18
06	Advanced Grid: Offsets, Ordering, Alignment & Nesting	20

PART III — DESIGN FOUNDATIONS

07	Typography	23
08	Color System & Theming	26
09	The Utility Engine: Spacing, Sizing & the Utility API	29
10	Flexbox Utilities & Layout Patterns	33

PART IV — CONTENT & FORMS

11	Images, Tables & Figures	36
12	Forms — Building Blocks	39
13	Forms — Validation & Advanced Patterns	43

PART V — COMPONENTS

14	Buttons, Badges, Alerts & Cards	46
15	Navbar, Navs, Tabs & Dropdowns	49
16	Modals, Offcanvas, Collapse & Accordion	52

17	Carousel, Tooltips, Popovers, Toasts & Spinners	56
18	Pagination, Progress, List Groups & Placeholders	59

PART VI — GOING PROFESSIONAL

19	Helper Classes	62
20	Customizing Bootstrap with Sass & Dark Mode	65
21	Design Principles: Making Bootstrap Look Custom	69
22	Performance, Accessibility & Best Practices	72
23	Capstone Project: A Complete Landing Page	75
24	Cheat Sheet & Class Reference	81

PART I • FOUNDATIONS

Welcome to Bootstrap 5

What Bootstrap actually is, why it still matters in 2026, and the mental model you need before writing a single line of markup.

1.1 What Bootstrap Is, Structurally

Bootstrap is not a "design." It is a **system of pre-written CSS rules and small JavaScript behaviors**, each one attached to a class name, that you compose on top of plain HTML. When you write `<div class="container">`, you are not asking Bootstrap to draw a box for you — you are opting your element into a rule that already exists in a stylesheet, one that says "give this element a responsive max-width and horizontal padding." Every visual result you get from Bootstrap is the sum of small, named, reusable rules like this one.

Structurally, Bootstrap is organized into three layers, and understanding this layering is the single most useful thing you can learn before writing any code:

1. **Reboot** — a normalization layer that resets browser inconsistencies (margins, font inheritance, box-sizing) so every browser starts from the same baseline.
2. **Layout & Utilities** — the grid system and thousands of single-purpose utility classes (spacing, color, display, flex, position) that let you build layouts without writing custom CSS.
3. **Components** — pre-assembled UI patterns (navbars, cards, modals, carousels) built by combining Reboot and utility classes, optionally enhanced with JavaScript for interactivity (dropdowns opening, modals appearing, carousels rotating).

Every chapter in this book maps to one of these three layers. Once that map is in your head, Bootstrap stops feeling like a giant class-name dictionary to memorize and starts feeling like a small number of composable rules you can predict.

NOTE

Bootstrap 5 removed the jQuery dependency entirely. All interactive components (modals, dropdowns, tooltips, carousels) are powered by plain JavaScript. You can use Bootstrap's CSS without ever loading its JavaScript file if you only need layout and styling.

1.2 Why Choose Bootstrap Over Writing Custom CSS

A fair question, especially now that CSS Grid and Flexbox are natively powerful. The honest answer is that Bootstrap trades a small amount of file size and a "house style" you must consciously override, for three things that are expensive to build yourself:

- **A tested responsive grid** that behaves identically across six breakpoints and every major browser, including edge cases like gutter collapsing and nested columns.
- **A consistent design token system** — spacing, color, typography, and radius scales — so that a button, a card, and a form input all agree on what "medium spacing" or "primary color" means.
- **Accessible interactive components** with correct ARIA attributes and keyboard behavior already implemented, which is easy to get wrong from scratch.

The skill this book teaches is not "memorize class names." It is: understand the underlying system well enough that you can predict what a class does before you look it up, and know when to override Bootstrap's defaults so your site doesn't look like every other Bootstrap site.

1.3 What "Beautiful" Means With Bootstrap

The most common criticism of Bootstrap is that sites built with it "look like Bootstrap" — recognizable rounded cards, the same blue buttons, the same navbar. This is a choice, not a limitation. Every value you see by default (the purple-blue primary color, the 0.375rem border radius, the specific font stack) is a Sass variable with a default value, not a hard-coded rule. Chapter 20 shows you exactly how to change every one of these defaults. The goal of this book is that by the final chapter, you can produce a page that uses Bootstrap's engineering underneath while looking nothing like a "default Bootstrap site" on the surface.

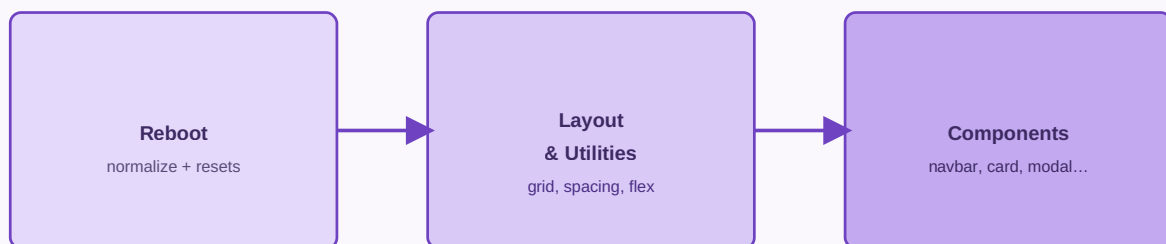


Figure 1.1 — Bootstrap's three structural layers. Each layer builds on the one before it.

1.4 How This Book Is Organized

Part I gets Bootstrap running and explains its foundational assumptions. Part II is dedicated entirely to the grid, including the arithmetic behind column widths and container sizes — this is the single most important system to

internalize. Part III covers the design-token systems: typography, color, spacing, and Flexbox utilities. Part IV covers forms and content. Part V is a complete tour of every component, grouped logically. Part VI is where you move from "using Bootstrap" to "art-directing Bootstrap": Sass customization, dark mode, professional design judgment, performance, and a full capstone project that uses everything from the previous 22 chapters to build one real, polished landing page from an empty file.

Getting Bootstrap Into Your Project

Three ways to install Bootstrap, what each file actually contains, and which approach to pick for a learning project versus a production site.

2.1 Method A — CDN (fastest way to start)

The Content Delivery Network method requires no build tools. You link two files: the compiled CSS in the `<head>`, and the bundled JavaScript just before the closing `</body>` tag. The "Bundle" build of the JS includes Popper.js, which Bootstrap needs internally to position tooltips, popovers, and dropdowns.

HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My Page</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>

  <h1 class="text-primary text-center mt-5">It works!</h1>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/boot-
strap.bundle.min.js"></script>
</body>
</html>
```

The `viewport` meta tag is not optional decoration — it is what tells mobile browsers to render the page at device width instead of faking a desktop-sized viewport and zooming out. Without it, none of Bootstrap's responsive breakpoints behave correctly on a phone.

2.2 Method B — npm (for build-tool projects)

TERMINAL

```
npm install bootstrap@5.3.3 @popperjs/core
```

With a bundler (Vite, Webpack, Parcel) you import the compiled CSS and JS directly into your application's entry file, letting the bundler package everything together:

JAVASCRIPT

```
import 'bootstrap/dist/css/bootstrap.min.css';  
import 'bootstrap/dist/js/bootstrap.bundle.min.js';
```

This is the right choice the moment your project has more than a couple of pages, because it lets you move to Method C (source Sass) later without changing how you consume Bootstrap.

2.3 Method C — Sass Source (for real customization)

This is the method every production, professionally-designed Bootstrap site actually uses, and it's the method Chapter 20 builds on in depth. Instead of importing the pre-compiled CSS, you import Bootstrap's raw `.scss` source files and compile them yourself, which lets you override design-token variables *before* Bootstrap uses them.

CUSTOM.SCSS

```
// 1. Override default variables BEFORE importing Bootstrap  
$primary: #0d6efd;  
$border-radius: .5rem;  
$font-family-base: 'Inter', sans-serif;  
  
// 2. Import Bootstrap's source  
@import "bootstrap/scss/bootstrap";
```

TIP

Variable overrides must come *before* the `@import` line. Sass resolves `!default` variables top-down; Bootstrap's own variables are declared with `!default` precisely so that anything you set earlier in the file wins.

2.4 Which Method Should You Use?

SITUATION	RECOMMENDED METHOD	WHY
Learning, quick prototype, single page	CDN	Zero setup, works instantly
Multi-page app with a bundler already	npm (compiled)	Version-locked, works offline, tree-shakeable JS
Client site, agency work, custom brand	Sass source	Full control of every design token, smaller output CSS

2.5 Verifying Your Setup

A reliable smoke test: add `class="container mt-5"` to a div and put a `class="btn btn-primary"` button inside it. If the button is purple-blue, rounded, and has padding — and the container is centered with margins on a wide screen — your CSS is loading. Then click nothing yet; components requiring JavaScript (dropdowns, modals) will silently do nothing if the JS bundle failed to load, which is a common early debugging trap: check your browser console for a 404 on the script tag before assuming your markup is wrong.

How Bootstrap Thinks: Reboot, Box-Sizing & Breakpoints

The invisible foundation every other chapter assumes you already understand.

3.1 Reboot: Fixing the Browser's Defaults

Every browser ships its own default stylesheet, and they disagree with each other in small but annoying ways — different default margins on headings, different form control fonts, different list indentation. Bootstrap's **Reboot** module is a single opinionated stylesheet that runs after the browser defaults and normalizes these inconsistencies. You don't call Reboot directly; it's included automatically the moment you load `bootstrap.css`. The most important thing Reboot does, structurally, is this one rule:

CSS (CONCEPTUAL — THIS IS WHAT REBOOT SETS)

```
*, *::before, *::after {  
  box-sizing: border-box;  
}
```

3.2 The Box-Sizing Rule and Why the Math Depends On It

THE MATH

By default, CSS uses `box-sizing: content-box`, where an element's declared width describes only the content area — padding and border are added *on top*. A 200px-wide box with 20px padding and a 2px border actually occupies 244px. Bootstrap switches every element to `border-box`, where the declared width already *includes* padding and border. That same 200px box, with the same padding and border, still occupies exactly 200px on screen.

This single switch is why Bootstrap's grid math (Chapter 4) works cleanly. A column that is told to be "33.333% wide" stays exactly 33.333% wide including its internal padding, instead of silently growing past its container and breaking the row. Without `border-box`, every percentage-based grid system in CSS would need constant manual correction.

content-box (browser default)



border-box (Bootstrap's Reboot)

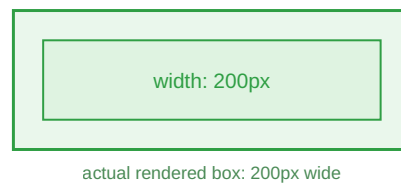


Figure 3.1 — Same declared width, same padding and border, different real-world size.

3.3 The Six Breakpoints

Bootstrap's entire responsive system — grid, visibility utilities, spacing, flex direction — is built around six fixed viewport-width thresholds called **breakpoints**. Each one has a short code used as a class-name "infix":

BREAKPOINT	INFIX	MIN WIDTH	TYPICAL DEVICE
Extra small	(none)	<576px	Small phones (portrait)
Small	<code>sm</code>	≥576px	Large phones (landscape)
Medium	<code>md</code>	≥768px	Tablets
Large	<code>lg</code>	≥992px	Laptops
Extra large	<code>xl</code>	≥1200px	Desktops
Extra extra large	<code>xxl</code>	≥1400px	Large monitors

Notice that the base (extra-small) tier has no infix — a class like `col-4` or `p-3` with no breakpoint code applies at *every* width, starting from zero, unless overridden by a larger breakpoint. This is the mobile-first principle described next.

3.4 Mobile-First: min-width, Not max-width

Bootstrap's media queries are all written with `min-width`, never `max-width`. That means every rule is a floor, not a ceiling: `col-md-6` means "become 6/12 columns wide once the viewport is *at least* 768px — and before that, fall back to whatever smaller-breakpoint class is set, or 100% width if none is set." Classes stack upward and the last matching rule (the widest breakpoint whose condition is true) wins, because later media queries in the stylesheet override earlier ones at the same specificity.

HTML

```
<div class="col-12 col-md-6 col-lg-4">...</div>
```

Read this like a sentence: "Be 12 columns wide by default (full width, for phones). Once we reach medium screens, become 6 columns wide (half). Once we reach large screens, become 4 columns wide (a third)." You are always designing for the smallest screen first and adding rules that apply as the screen grows — never the reverse.

COMMON MISTAKE

Writing `col-lg-4 col-md-6 col-12` in that order in your *class list* has no effect on behavior — CSS class order in HTML never matters, only the specificity and source order of the underlying stylesheet rules do. But writing them in ascending breakpoint order in your own head and code keeps your mental model (and your team's) consistent.

3.5 CSS Custom Properties (Variables) in Bootstrap 5

Since version 5.2, most Bootstrap components expose their design tokens as CSS custom properties (variables) scoped with a `--bs-` prefix, directly on the element. You can inspect any button in your browser's dev tools and see something like `--bs-btn-padding-x`. This means you can override a single instance of a component without writing a new CSS class or touching Sass at all:

HTML

```
<button class="btn btn-primary" style="--bs-btn-border-radius: 2rem;">
  Pill Button
</button>
```

This "CSS variable override" technique is the fastest way to make one-off tweaks in Chapter 21 without recompiling Sass.

PART II • THE GRID SYSTEM

The 12-Column Grid: Concept & Math

The exact arithmetic behind every column width, so you never have to guess again.

4.1 The Three-Level Structure

Every Bootstrap grid layout is built from exactly three nested levels, and this nesting is not optional — skipping a level breaks the math:

1. `.container` (or `.container-fluid`) — establishes the horizontal boundary and centers content.
2. `.row` — a flex container that cancels out the container's padding using negative margins, and holds columns.
3. `.col-*` — the actual content-holding boxes, sized as a fraction of 12.

HTML

```
<div class="container">
  <div class="row">
    <div class="col">Column A</div>
    <div class="col">Column B</div>
    <div class="col">Column C</div>
  </div>
</div>
```

With no numbers specified, plain `.col` classes split the available space *equally* — three `.col` elements each become exactly one-third of the row, automatically, because they're all set to `flex: 1 1 0%` under the hood.

4.2 Why 12? The Divisibility Argument

THE MATH

12 is chosen because it has more integer divisors than almost any small number: 1, 2, 3, 4, 6, and 12. This means a 12-column grid can be split evenly into halves (6+6), thirds (4+4+4), quarters (3+3+3+3), or sixths (2×6) without any fractional column widths. A 10-column grid, by contrast, cannot be split into thirds without a repeating decimal. This is purely arithmetic — nothing about "12" is special except how many small integers divide into it evenly.

4.3 The Column Width Formula

Every numbered column class follows one formula:

THE MATH

$\text{width} = (\text{span} \div 12) \times 100\%$

So `.col-4` computes to $(4 \div 12) \times 100\% = 33.3333\%$. `.col-8` computes to $(8 \div 12) \times 100\% = 66.6667\%$. Because these two numbers sum to exactly 100%, a `.col-4` and a `.col-8` sit perfectly side by side in one row with no gap or overflow.

CLASS	SPAN	FORMULA	COMPUTED WIDTH
<code>.col-1</code>	1	$1/12 \times 100$	8.3333%
<code>.col-2</code>	2	$2/12 \times 100$	16.6667%
<code>.col-3</code>	3	$3/12 \times 100$	25%
<code>.col-4</code>	4	$4/12 \times 100$	33.3333%
<code>.col-6</code>	6	$6/12 \times 100$	50%
<code>.col-8</code>	8	$8/12 \times 100$	66.6667%
<code>.col-9</code>	9	$9/12 \times 100$	75%
<code>.col-12</code>	12	$12/12 \times 100$	100%

4.4 Combining Columns: The Row Must Sum to 12 (or Wrap)

You are not required to make every row sum to exactly 12 — but if it exceeds 12, the excess columns wrap onto a new line, because internally each column is a flex item with `flex-wrap: wrap` set on the row.

HTML

```
<div class="row">
  <div class="col-8">Main content (8/12 = 66.67%)</div>
  <div class="col-4">Sidebar (4/12 = 33.33%)</div>
  <div class="col-6">This wraps to a new line – 8+4+6 = 18, which exceeds 12</div>
</div>
```

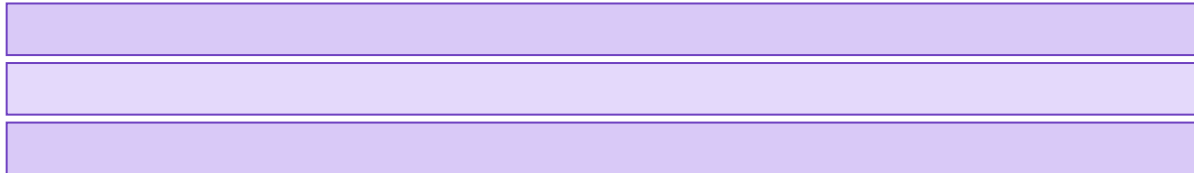
4.5 Responsive Columns: Different Fractions at Different Breakpoints

The infix system from Chapter 3 applies directly to columns. `col-{breakpoint}-{span}` means "become this fraction once we reach this breakpoint or wider."

HTML

```
<div class="row">
  <div class="col-12 col-md-4">Stacks full-width on phones,
    becomes 1/3 on tablets and up</div>
  <div class="col-12 col-md-4">Same</div>
  <div class="col-12 col-md-4">Same</div>
</div>
```

Below 768px (mobile) — col-12 stacks



At 768px and above — col-md-4 gives thirds

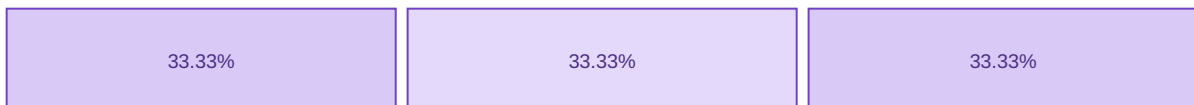


Figure 4.1 — The same three `col-12 col-md-4` elements at two different viewport widths.

4.6 Auto-Layout Columns

Plain `.col` (with no number) auto-sizes to share space equally with sibling `.col` elements. Mix it with a numbered sibling and the auto column takes whatever width is left over:

HTML

```
<div class="row">
  <div class="col-3">Fixed: 25%</div>
  <div class="col">Fills remaining 75%</div>
</div>
```

4.7 Row-Columns: Setting Child Width From the Parent

Instead of putting a width class on every single column, `.row-cols-{n}` on the *row* tells all direct children how many columns to display per row — useful for card grids and galleries.

HTML

```
<div class="row row-cols-1 row-cols-sm-2 row-cols-lg-4 g-3">
  <div class="col"><div class="card">Item 1</div></div>
  <div class="col"><div class="card">Item 2</div></div>
  <div class="col"><div class="card">Item 3</div></div>
  <div class="col"><div class="card">Item 4</div></div>
</div>
```

Here, `row-cols-1` means one card per row on phones, `row-cols-sm-2` means two per row from 576px up, and `row-cols-lg-4` means four per row from 992px up — this is exactly how most product grids and photo galleries are built in practice.

4.8 The Gutter: Why Columns Don't Touch

THE MATH

The default gutter width is `1.5rem` (24px), stored as the CSS variable `--bs-gutter-x`. This total is split in half: each column gets `calc(var(--bs-gutter-x) * 0.5)` — 12px — as *padding* on both its left and right side. The row itself then applies `-12px` negative margins on its own left and right edges. This cancels the first and last column's outer padding so the row's total content still lines up flush with the container's edges, while the `12px + 12px = 24px` gutter appears cleanly *between* adjacent columns.

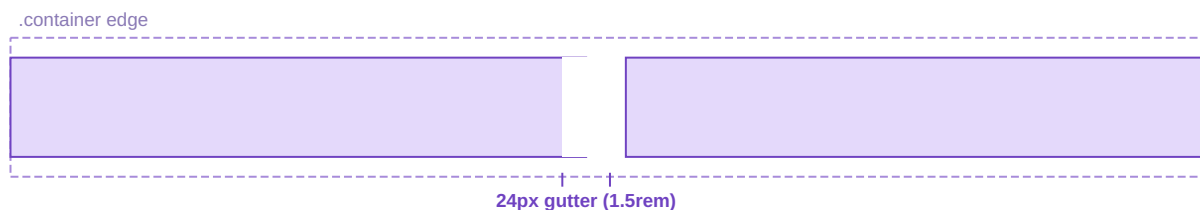


Figure 4.2 — Two `col-6` elements with the default 24px gutter shown as the white gap between them.

4.9 Controlling Gutters With Utility Classes

The `.g-*`, `.gx-*`, and `.gy-*` classes (added to the row) resize gutters using the same 0–5 spacing scale from Chapter 9: `g-0` removes gutters entirely (columns touch), `gx-5` sets only horizontal gutters to the largest step, and `gy-4` sets vertical gap between wrapped rows of columns.

HTML

```
<div class="row g-0"> <!-- no gutters, columns touch exactly -->
<div class="row gx-5"> <!-- extra-wide horizontal gutter only -->
<div class="row gy-4"> <!-- adds vertical spacing between wrapped rows -->
```

Containers Explained

Three container types, and the exact max-width table you need memorized.

5.1 The Three Container Classes

CLASS	BEHAVIOR
<code>.container</code>	Fixed max-width that changes at each breakpoint (see table below); centered with <code>margin: 0 auto</code> .
<code>.container-fluid</code>	Always <code>100%</code> width, at every breakpoint. Only horizontal padding is applied, never a max-width cap.
<code>.container-{breakpoint}</code>	Behaves as <code>100%</code> wide <i>until</i> the named breakpoint, then locks to a fixed max-width from that point up. E.g. <code>.container-lg</code> is fluid below 992px, fixed above it.

5.2 The Container Max-Width Table

THE MATH

Each fixed max-width is deliberately set slightly *below* its matching breakpoint's viewport width — this guarantees a visible margin of "breathing room" on the sides even at the exact breakpoint boundary, rather than the container touching the viewport edge.

BREAKPOINT	VIEWPORT ≥	.CONTAINER MAX-WIDTH
sm	576px	540px
md	768px	720px
lg	992px	960px
xl	1200px	1140px
xxl	1400px	1320px

Below 576px, `.container` is simply `width: 100%` with `1.5rem` (equal to half a gutter $\times$ 2, i.e. `0.75rem` per side... actually the container's own side padding is `calc(var(--bs-gutter-x) * 0.5)`, the same 12px per side used by columns) of horizontal padding — full-bleed with a comfortable inset on phones.

5.3 Choosing the Right Container

- Use plain `.container` for standard marketing pages and blog content — text and content that should never stretch uncomfortably wide on a large monitor.
- Use `.container-fluid` for dashboards, admin panels, or full-width hero sections and image bands that should bleed edge-to-edge on any screen.
- Use `.container-{bp}` when you want a hybrid — for example a page that should feel edge-to-edge on tablets but gain a comfortable reading width on large desktop monitors.

HTML

```
<div class="container-fluid p-0">
  
</div>
<div class="container mt-5">
  <p>Regular body copy, comfortably capped in width for readability.</p>
</div>
```

5.4 A Common Professional Pattern: Nested Containers

A frequent real-world pattern for hero sections is a fluid outer section for a full-width background color or image, with a regular fixed-width container nested inside it to keep the text readable:

HTML

```
<section class="bg-dark text-white">
  <div class="container py-5">
    <h1 class="display-4">Full-width background, readable text</h1>
    <p class="lead">The dark background bleeds edge to edge, while this
      paragraph stays within a comfortable max-width.</p>
  </div>
</section>
```

Advanced Grid: Offsets, Ordering, Alignment & Nesting

Everything beyond basic column widths — the tools that let you build genuinely complex layouts.

6.1 Offsets: Pushing Columns With Margin Math

THE MATH

`.offset-{n}` applies `margin-left: (n ÷ 12) × 100%`. It is exactly the same formula as column width, just applied as a left margin instead of an element width — which is why offsets and column spans always combine predictably: an offset column's total footprint is `offset-n + col-m`, and as long as that sum doesn't exceed 12, it fits in one row.

HTML

```
<div class="row">
  <div class="col-4 offset-4">Centered 4-wide column (4 + 4 offset + 4 leftover space = 12)</div>
</div>
```

This is the standard way to center a fixed-width column without Flexbox alignment utilities — commonly used for centering a login form or a single call-to-action column.

6.2 Reordering Columns Visually Without Changing HTML

`.order-{0-5}` (plus `.order-first` and `.order-last`) reassigns the CSS `order` flex property, letting the visual sequence differ from the source-code sequence — useful for putting an image after text in your HTML (good for reading order and accessibility) but displaying it first on desktop.

HTML

```
<div class="row">
  <div class="col-md-6 order-md-2">Text (appears second on desktop)</div>
  <div class="col-md-6 order-md-1">Image (appears first on desktop)</div>
</div>
```

6.3 Vertical Alignment: `align-items` and `align-self`

Because a `.row` is a flex container, standard Flexbox alignment properties are exposed as utility classes. Put `.align-items-*` on the **row** to align all columns at once; put `.align-self-*` on an individual **column** to override just that one.

CLASS	EFFECT ON THE CROSS-AXIS (VERTICAL, IN A NORMAL ROW)
<code>.align-items-start</code>	Columns align to the top of the row
<code>.align-items-center</code>	Columns vertically centered
<code>.align-items-end</code>	Columns align to the bottom
<code>.align-items-stretch</code>	(default) columns stretch to match the tallest sibling

6.4 Horizontal Distribution: `justify-content`

When columns in a row don't add up to 12 (so they don't fill the full width), `.justify-content-*` on the row controls how the leftover horizontal space is distributed along the main axis.

HTML

```
<div class="row justify-content-between">
  <div class="col-3">Left</div>
  <div class="col-3">Right (pushed to the far edge)</div>
</div>
```

Options are `start`, `center`, `end`, `around`, `between`, and `evenly` — the last three differ in exactly how the gaps between and around items are calculated, with `between` putting all extra space strictly between items (none on the outer edges), `around` giving each item equal space on both sides (so gaps between items look twice as large as the outer edges), and `evenly` making every gap, including the outer edges, mathematically identical.

6.5 Nesting Rows Inside Columns

Any column can contain its own nested `.row`, which restarts the 12-column math *relative to that parent column's width*, not the page. A nested `col-6` is 50% of its parent column, not 50% of the page.

HTML

```
<div class="row">
  <div class="col-8">
    <div class="row">
      <div class="col-6">Nested A (50% of the 8-wide parent)</div>
      <div class="col-6">Nested B (50% of the 8-wide parent)</div>
    </div>
  </div>
  <div class="col-4">Sidebar</div>
</div>
```

TIP

You do not need to wrap a nested row in another `.container` — the nested `.row`'s negative margins automatically cancel out the parent column's padding, restoring a clean edge-to-edge 12-unit grid inside it.

6.6 Vertical and Horizontal Gutters Recap With Nesting

Since nested rows inherit the CSS custom property `--bs-gutter-x` unless you override it, applying `g-2` to an inner row lets you use a tighter gutter for a dense sub-layout (like a small icon grid) while keeping the outer page layout on the default, more generous gutter.

PART III · DESIGN FOUNDATIONS

Typography

Headings, body text, display type, and the utility classes that shape how words look on the page.

7.1 The Base Typographic Settings

Reboot sets the default body font stack to the operating system's native UI font (a "system font stack") at `1rem` (16px) with a `1.5` line-height. Headings `h1` – `h6` get a shared font-weight of `500` and a bottom margin of `0.5rem`, with sizes descending from `2.5rem` (`h1`) down to `1rem` (`h6`).

ELEMENT / CLASS	DEFAULT SIZE
<code>h1</code>	2.5rem (40px)
<code>h2</code>	2rem (32px)
<code>h3</code>	1.75rem (28px)
<code>h4</code>	1.5rem (24px)
<code>h5</code>	1.25rem (20px)
<code>h6</code>	1rem (16px)

7.2 Display Headings

For hero sections and marketing pages, regular headings are often too small. Bootstrap provides six **display** classes (`.display-1` through `.display-6`) — larger, bolder, with tighter line-height, meant for one big statement rather than document structure. They can be applied to any element, not just headings.

HTML

```
<h1 class="display-3 fw-bold">Build faster.</h1>
<p class="lead">A larger, lighter-weight introductory paragraph style,
  used right under a headline.</p>
```

THE MATH — RESPONSIVE FONT SIZING (RFS)

Bootstrap compiles its display headings and several other large text styles through an engine called **RFS (Responsive Font Sizes)**. Instead of one fixed `font-size`, RFS outputs a `calc()` expression combining a fixed `rem` value with a small amount of `vw` (viewport-width) unit, for example `font-size: calc(1.375rem + 1.5vw)`; . The `vw` term means the text genuinely gets a few pixels smaller as the browser narrows, in addition to whatever breakpoint-based rule already applies — so a `3rem` headline on a `1400px` screen doesn't feel oversized on a `380px` phone, without you writing a single manual media query for font size.

7.3 Inline Text Utilities

CLASS	EFFECT
<code>.fw-bold</code> / <code>.fw-normal</code> / <code>.fw-light</code>	font-weight 700 / 400 / 300
<code>.fst-italic</code> / <code>.fst-normal</code>	italic / normal font style
<code>.text-decoration-underline</code>	underline text
<code>.text-uppercase</code> / <code>.text-lowercase</code> / <code>.text-capitalize</code>	text-transform variants
<code>.text-truncate</code>	single-line ellipsis overflow (requires <code>display:block</code> or <code>inline-block</code>)
<code>.text-muted</code>	lighter gray, de-emphasized text
<code>.text-start</code> / <code>.text-center</code> / <code>.text-end</code>	text-align, responsive with infixes: <code>.text-md-start</code>
<code>.fs-1</code> – <code>.fs-6</code>	Applies the corresponding heading's font-size to any element
<code>.lh-1</code> / <code>.lh-sm</code> / <code>.lh-base</code> / <code>.lh-lg</code>	line-height presets

7.4 Lists Without Bullets, Blockquotes, and Abbreviations

HTML

```
<ul class="list-unstyled">
  <li>No bullets, no default padding</li>
  <li>Useful for building nav menus or footers from a <ul></li>
</ul>

<blockquote class="blockquote">
  <p>A larger, styled quotation.</p>
  <footer class="blockquote-footer">Cited in <cite>A Source</cite></footer>
</blockquote>
```

7.5 Choosing Fonts That Don't Look "Default Bootstrap"

The single fastest visual signature that gives away an un-customized Bootstrap site is its default system font stack. Swapping `$font-family-base` in Sass (Chapter 20) for a distinctive pairing — for example a geometric sans-serif for headings and a humanist sans or serif for body copy — does more to make a page feel custom-designed than almost any other single change.

TIP

When pairing two fonts, keep body copy in a highly legible, fairly neutral typeface, and reserve a more distinctive display face for headings only. Using a decorative font for paragraph text hurts readability and looks amateurish; using it only for short headlines showcases its personality without cost.

Color System & Theming

The theme-color map that powers every button, alert, badge, and text-color utility in Bootstrap.

8.1 The Theme Color Map

Almost every colored element in Bootstrap — buttons, alerts, badges, borders, text — pulls from one central Sass map called `$theme-colors`. This is the single most important customization point in the entire framework, because changing one entry here regenerates dozens of classes automatically.

NAME	DEFAULT HEX	TYPICAL USE
primary	#0d6efd	Main brand actions, primary buttons, links
secondary	#6c757d	Neutral, less prominent actions
success	#198754	Confirmations, positive states
danger	#dc3545	Errors, destructive actions
warning	#ffc107	Caution states
info	#0dcaf0	Neutral informational messages
light	#f8f9fa	Light backgrounds
dark	#212529	Dark backgrounds, high-contrast text

Each name generates a predictable family of classes: `.btn-primary`, `.text-primary`, `.bg-primary`, `.border-primary`, `.alert-primary`, `.list-group-item-primary`, and more — all built from the same one hex value.

HTML

```
<button class="btn btn-success">Save</button>
<span class="badge bg-danger">3 errors</span>
<p class="text-info">Informational note.</p>
<div class="border border-warning p-3">Bordered box</div>
```

8.2 Background Utilities and Text Contrast

Background color utilities (`.bg-*`) do *not* automatically change text color to remain readable — you are responsible for pairing a dark background with light text, typically with `.text-white` or the built-in `.text-bg-*` combined classes introduced in Bootstrap 5.3, which set both background and a correctly contrasting foreground color in one class.

HTML

```
<div class="text-bg-dark p-4 rounded">
  Background AND readable text color set together.
</div>
```

8.3 Subtle / Soft Color Variants

Bootstrap 5.3 introduced tint/shade variants for softer UI accents — a light-tinted background paired with a dark, matching-hue text color, popular in modern "soft" badge and alert designs.

HTML

```
<span class="badge" style="background-color: var(--bs-primary-bg-subtle);
  color: var(--bs-primary-text-emphasis);">Soft badge</span>
```

These are exposed as CSS variables (`--bs-{color}-bg-subtle` , `--bs-{color}-text-emphasis` , `--bs-{color}-border-subtle`) for every theme color, generated automatically from the base color using color-mixing functions in Sass — you never have to pick these tints and shades by eye.

8.4 Link Colors

Bootstrap's `.link-*` utilities (`.link-primary` , `.link-danger` , etc.) recolor an anchor tag and automatically adjust its hover/focus state to a slightly darker shade — useful for inline text links that need to signal meaning (like a "danger" delete link) without looking like a button.

8.5 Practical Color Theory for a Bootstrap Site

TIP

Resist the urge to use every theme color on one page. A disciplined, professional-looking palette typically uses: one primary brand color for calls-to-action and links, one neutral gray scale for text and backgrounds, and reserves success/danger/warning strictly for their semantic meaning (confirmations, errors, cautions) — never as decorative accents. This restraint alone makes a Bootstrap site look considerably more designed.

8.6 Color Modes: Light and Dark Themes

Since version 5.3, Bootstrap ships built-in support for color modes via the `data-bs-theme` attribute, switching an entire subtree of the page between light and dark palettes without a separate stylesheet.

HTML

```
<html data-bs-theme="dark">
  <!-- every component below now uses dark-mode colors automatically -->
</html>

<div data-bs-theme="light" class="p-3">
  This section stays light even inside a dark page.
</div>
```

Under the hood this works because Bootstrap's colors are CSS custom properties, and the dark mode selector simply redefines those variables' values within the scope of the `data-bs-theme="dark"` attribute — the actual component CSS (a card, a button) never changes; only the variables it reads from do. Chapter 20 covers building a fully custom dark theme.

The Utility Engine: Spacing, Sizing & the Utility API

How thousands of tiny classes are generated from a handful of Sass maps, and the spacing scale you'll use more than any other feature in this book.

9.1 The Naming Formula

Nearly every spacing class follows one predictable pattern:

THE PATTERN

`{property}{sides}-{breakpoint}-{size}`

Breakpoint is optional (omit for "all sizes"). For example `mt-md-3` reads as: **m**argin, **t**op side, at the **md** breakpoint and up, size step **3**.

PROPERTY CODE		MEANING
m		margin
p		padding

SIDE CODE	MEANING
t	top
b	bottom
s	start (left in LTR languages — logical property, flips automatically in RTL)
e	end (right in LTR)
x	left and right
y	top and bottom
(blank)	all four sides

9.2 The Spacing Scale — the Actual Numbers

THE MATH

Every spacing step is a multiple of the base `$spacer` variable, which defaults to `1rem` (16px). The scale is *not* perfectly linear — it roughly doubles at the high end to give you a genuinely large "5" step without needing end-less intermediate classes.

STEP	VALUE	PIXELS (AT 16PX ROOT)
0	0	0px
1	0.25rem	4px
2	0.5rem	8px
3	1rem	16px
4	1.5rem	24px
5	3rem	48px
auto	auto	used for centering with margin, e.g. <code>mx-auto</code>

HTML

```
<div class="p-4 mb-3">24px padding on all sides, 16px margin below</div>
<div class="mx-auto" style="width:200px;">Horizontally centered box</div>
<div class="px-2 px-md-5">8px horizontal padding on phones, 48px on tablets+</div>
```

9.3 Negative Margins

Prefixing the size with `n` produces a negative value: `mt-n3` is `margin-top: -1rem`. Negative margins are commonly used to intentionally overlap two elements, such as pulling an avatar image up over the edge of a card header.

9.4 Sizing Utilities

CLASS	EFFECT
<code>.w-25</code> / <code>.w-50</code> / <code>.w-75</code> / <code>.w-100</code> / <code>.w-auto</code>	width as a percentage of the parent, or auto
<code>.h-25</code> / <code>.h-50</code> / <code>.h-75</code> / <code>.h-100</code> / <code>.h-auto</code>	height, same scale
<code>.mw-100</code>	max-width: 100% — the classic "responsive image never overflows its container" rule
<code>.vw-100</code> / <code>.vh-100</code>	100% of the viewport width / height (not the parent)
<code>.min-vh-100</code>	minimum height of the full viewport — common for a full-height hero or a sticky-footer layout

9.5 The Utility API: How These Classes Are Actually Generated

Since Bootstrap 5.1, all utility classes — spacing, color, display, borders, and more — are generated from one large Sass map called `$utilities`, rather than being hand-written CSS rules. Each entry in the map describes a property, which responsive breakpoints to generate, and which values to loop over.

SIMPLIFIED STRUCTURE OF \$UTILITIES (SASS)

```
$utilities: (
  "opacity": (
    property: opacity,
    values: (0, 25, 50, 75, 100)
  ),
  ...
);
```

This is powerful because it means you can **add your own utility**, or modify an existing one, with a small Sass snippet rather than writing raw CSS classes by hand — and your new utility automatically gets responsive breakpoint variants for free.

CUSTOM.SCSS

```
@import "bootstrap/scss/functions";
@import "bootstrap/scss/variables";
@import "bootstrap/scss/maps";

// Add a new "cursor" utility: generates .cursor-pointer, .cursor-default, etc.
$utilities: map-merge($utilities, (
  "cursor": (
    property: cursor,
    values: pointer default grab not-allowed
  )
));

@import "bootstrap/scss/utilities";
@import "bootstrap/scss/utilities/api";
```

NOTE

This is genuinely advanced usage, revisited in full in Chapter 20. For now, the important takeaway is structural: every utility class you type is a lookup into this one map, which is why the naming pattern is so consistent across hundreds of classes — they were all generated by the same code.

Flexbox Utilities & Layout Patterns

Bootstrap doesn't have a separate "layout engine" beyond the grid — most real layout problems are solved with a handful of flex utility classes applied directly to any element.

10.1 Turning Any Element Into a Flex Container

HTML

```
<div class="d-flex">...</div>      <!-- display: flex -->
<div class="d-inline-flex">...</div>  <!-- display: inline-flex -->
```

You do not need `.row` to use Flexbox — `.d-flex` works on any element and is the basis for most component internals (a card's footer, a navbar's link list) as well as everyday layout problems like aligning an icon next to text.

10.2 Direction and Wrapping

CLASS	EFFECT
<code>.flex-row</code> (default)	children laid out left to right
<code>.flex-column</code>	children stacked top to bottom
<code>.flex-row-reverse</code> / <code>.flex-column-reverse</code>	reversed order, same axis
<code>.flex-wrap</code> / <code>.flex-nowrap</code>	allow or forbid wrapping to new lines

All of these accept responsive infixes: `.flex-column.flex-md-row` is an extremely common pattern — stack vertically on phones, lay out horizontally from tablets up — used constantly for card layouts and toolbars.

10.3 Alignment: the Two-Axis Model

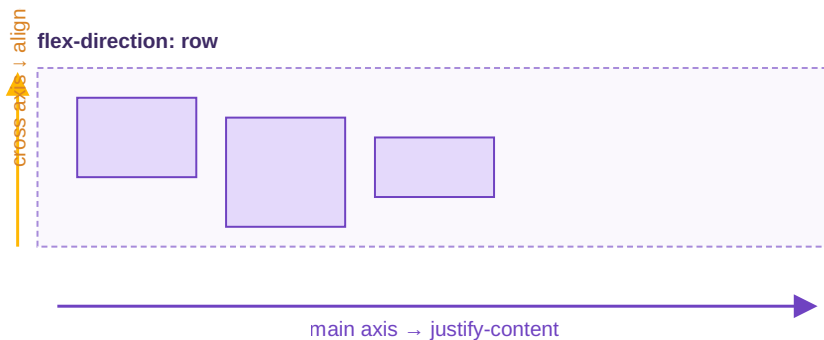


Figure 10.1 — *justify-content* always controls the main axis; *align-items* always controls the cross axis. When you switch to *flex-column*, the two axes swap roles.

CLASS FAMILY	CONTROLS	VALUES
<code>.justify-content-*</code>	Main-axis distribution	start, end, center, between, around, evenly
<code>.align-items-*</code>	Cross-axis alignment of all children	start, end, center, baseline, stretch
<code>.align-self-*</code>	Cross-axis alignment of one child, overriding <code>align-items</code>	same values
<code>.align-content-*</code>	Spacing between wrapped lines (only visible with <code>flex-wrap</code> and extra height)	start, end, center, between, around, stretch

COMMON MISTAKE

`align-items` and `align-content` are frequently confused. `align-items` aligns items *within* their line; `align-content` only has a visible effect when there are *multiple wrapped lines* and extra free space on the cross axis to distribute between those lines. On a single-line flex container, `align-content` does nothing.

10.4 Growing, Shrinking, and Ordering

HTML

```
<div class="d-flex">
  <div class="flex-shrink-0">Logo (never shrinks)</div>
  <div class="flex-grow-1">Nav links (fills remaining space)</div>
  <div class="flex-shrink-0">Button</div>
</div>
```

`.flex-fill` is shorthand equivalent to `flex: 1 1 auto`, commonly applied to every child in a row of equal-width Flexbox items where you don't want to think about the grid's 12-column math at all.

10.5 Stacks: The Simplified Flex Shortcut

`.vstack` and `.hstack` (covered fully as Helpers in Chapter 19) are thin convenience wrappers around `d-flex` plus a sensible default direction and gap — reach for them whenever a full grid row feels like overkill for a simple list of stacked or inline items.

10.6 A Real Layout Pattern: The "Holy Grail" Header

HTML

```
<header class="d-flex justify-content-between align-items-center p-3 border-bottom">
  <a class="fs-4 fw-bold text-decoration-none" href="#">Brand</a>
  <nav class="d-none d-md-flex gap-4">
    <a href="#">Product</a><a href="#">Pricing</a><a href="#">About</a>
  </nav>
  <button class="btn btn-primary">Sign up</button>
</header>
```

This single line — `d-flex justify-content-between align-items-center` — is arguably the most-used Flexbox combination in real Bootstrap projects: it pins one item to each end and vertically centers everything, which is exactly what a header, a card footer, or a list item with a trailing action button needs.

PART IV · CONTENT & FORMS

Images, Tables & Figures

The small content-level classes that keep media and data from breaking your layout.

11.1 Responsive Images

HTML

```

```

THE MATH

`.img-fluid` applies exactly two rules: `max-width: 100%` and `height: auto`. The image can never render wider than its parent container, and because height is set to `auto` rather than a fixed value, the browser recalculates height using the image's *natural aspect ratio* — width shrinks, height shrinks proportionally, so the picture never distorts.

11.2 Image Thumbnails and Shapes

HTML

```
    <!-- padded border + rounded corners -->
          <!-- soft corners -->
    <!-- perfect circle (needs equal width/height) -->
```

11.3 Figures

HTML

```
<figure class="figure">
  
  <figcaption class="figure-caption">Fig. 1 – Quarterly revenue.</figcaption>
</figure>
```

11.4 The Ratio Helper: Locking Aspect Ratio for Embeds

Embedding a video (an `<iframe>`) has historically been painful because iframes don't respect `height: auto` the way images do. The `.ratio` helper solves this with a classic CSS technique: a wrapper whose height is driven entirely by *padding*, calculated as a percentage of width.

THE MATH

For a 16:9 box, `padding-top` is set to $(9 \div 16) \times 100\% = 56.25\%$. Because percentage-based padding is always calculated relative to the element's own *width* (a quirk of the CSS box model), this padding value forces the box's height to always be exactly 56.25% of whatever width it happens to render at — at any screen size, with no JavaScript and no fixed pixel values.

HTML

```
<div class="ratio ratio-16x9">
  <iframe src="https://www.youtube.com/embed/xyz" allowfullscreen></iframe>
</div>
```

CLASS	RATIO	PADDING-TOP
<code>.ratio-1x1</code>	1:1	100%
<code>.ratio-4x3</code>	4:3	75%
<code>.ratio-16x9</code>	16:9	56.25%
<code>.ratio-21x9</code>	21:9	42.857%
<code>.ratio</code> + inline <code>style="--bs-aspect-ratio: 42%"</code>	custom	any value you supply

11.5 Tables

HTML

```
<table class="table table-striped table-hover table-bordered align-middle">
  <thead class="table-dark">
    <tr><th>#</th><th>Name</th><th>Status</th></tr>
  </thead>
  <tbody>
    <tr><td>1</td><td>Ada</td><td><span class="badge bg-success">Active</span></td></tr>
    <tr><td>2</td><td>Grace</td><td><span class="badge bg-secondary">Invited</span></td></tr>
  </tbody>
</table>
```

CLASS	EFFECT
<code>.table</code>	Base spacing and border styling (required base class)
<code>.table-striped</code>	Alternating row background (zebra striping)
<code>.table-hover</code>	Highlights a row on mouse hover
<code>.table-bordered</code>	Border on every cell, not just horizontal dividers
<code>.table-border-less</code>	Removes all borders
<code>.table-sm</code>	Halves cell padding for a denser table
<code>.table-responsive</code>	Wraps the table in a scrollable container below a chosen breakpoint, so wide tables don't break mobile layouts

TIP

Always wrap wide data tables in `<div class="table-responsive">`. Without it, a table with many columns will force the entire page to scroll horizontally on a phone, which is one of the most common real-world Bootstrap layout bugs.

Forms — Building Blocks

Every input, select, checkbox, and switch, and how to lay them out with the grid.

12.1 The Base Control Classes

CLASS	APPLIES TO
<code>.form-control</code>	<code><input></code> (text, email, password, etc.) and <code><textarea></code>
<code>.form-select</code>	<code><select></code> dropdowns
<code>.form-check-input</code>	checkboxes and radios
<code>.form-range</code>	<code><input type="range"></code> sliders
<code>.form-label</code>	the associated <code><label></code>

HTML

```
<div class="mb-3">
  <label for="email" class="form-label">Email address</label>
  <input type="email" class="form-control" id="email" placeholder="name@example.com">
</div>

<div class="mb-3">
  <label for="country" class="form-label">Country</label>
  <select class="form-select" id="country">
    <option selected>Choose...</option>
    <option value="1">India</option>
    <option value="2">United States</option>
  </select>
</div>
```

12.2 Checkboxes, Radios, and Switches

HTML

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="terms">
  <label class="form-check-label" for="terms">I agree to the terms</label>
</div>

<div class="form-check form-switch">
  <input class="form-check-input" type="checkbox" role="switch" id="notify">
  <label class="form-check-label" for="notify">Email notifications</label>
</div>

<div class="form-check form-check-inline">
  <input class="form-check-input" type="radio" name="plan" id="planA" checked>
  <label class="form-check-label" for="planA">Monthly</label>
</div>
<div class="form-check form-check-inline">
  <input class="form-check-input" type="radio" name="plan" id="planB">
  <label class="form-check-label" for="planB">Annual</label>
</div>
```

NOTE

Adding `form-switch` alongside `form-check` is purely a visual transformation of a checkbox into a toggle-switch shape — it is still, semantically and functionally, a checkbox input. Adding `role="switch"` improves how assistive technology announces it.

12.3 Sizing and Disabled/Readonly States

HTML

```
<input class="form-control form-control-lg" placeholder="Large">
<input class="form-control" placeholder="Default">
<input class="form-control form-control-sm" placeholder="Small">
<input class="form-control" placeholder="Disabled" disabled>
<input class="form-control" value="Can't edit this" readonly>
```

12.4 Floating Labels

Floating labels overlay the label text inside the input box itself, animating it upward once the field has focus or content — a widely-used modern pattern for compact, polished-looking forms.

HTML

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="fEmail" placeholder="name@example.com">
  <label for="fEmail">Email address</label>
</div>
```

COMMON MISTAKE

Floating labels require the `placeholder` attribute to be present (even if empty, `placeholder=" "`) and the `<label>` to come *after* the input in the HTML source, because the effect relies on a CSS sibling selector reacting to the input's `:focus` and `:placeholder-shown` states. Swapping the order breaks the animation silently.

12.5 Input Groups

Input groups visually attach text, icons, or buttons to the side of an input, forming one seamless-looking control.

HTML

```
<div class="input-group mb-3">
  <span class="input-group-text">@</span>
  <input type="text" class="form-control" placeholder="Username">
</div>

<div class="input-group">
  <input type="text" class="form-control" placeholder="Search...">
  <button class="btn btn-primary" type="button">Go</button>
</div>
```

12.6 Laying Out Forms With the Grid

Because form controls are just block elements, they combine naturally with everything from Chapter 4 – 6. A common two-column form row:

HTML

```
<form class="row g-3">
  <div class="col-md-6">
    <label class="form-label">First name</label>
    <input class="form-control">
  </div>
  <div class="col-md-6">
    <label class="form-label">Last name</label>
    <input class="form-control">
  </div>
  <div class="col-12">
    <button class="btn btn-primary" type="submit">Submit</button>
  </div>
</form>
```

The **g-3** gutter class on the row is what creates comfortable vertical spacing between wrapped form rows on mobile, in addition to the horizontal gutter between the two columns on desktop.

Forms — Validation & Advanced Patterns

Giving users clear feedback, and the small set of patterns that make forms feel professional rather than generic.

13.1 Client-Side Validation Styling

Bootstrap doesn't perform validation logic itself — that's the browser's native HTML5 Constraint Validation API (`required` , `pattern` , `type="email"` , etc.) or your own JavaScript. What Bootstrap provides is the *visual styling* layered on top: green/red borders, icons, and feedback text, triggered by the presence of `.is-valid` / `.is-invalid` classes, or automatically via the `:valid` / `:invalid` pseudo-classes once a form has the `.was-validated` class.

HTML

```
<form class="was-validated">
  <div class="mb-3">
    <label class="form-label">Username</label>
    <input type="text" class="form-control" required>
    <div class="valid-feedback">Looks good!</div>
    <div class="invalid-feedback">Please choose a username.</div>
  </div>
</form>
```

NOTE

`.was-validated` is usually added by a small JavaScript snippet on the form's `submit` event (calling `form.checkValidity()` and then adding the class), not written into the HTML from page load — otherwise every field would show red before the user has even typed anything.

13.2 Manually Forcing a State

For server-side validation results (a username taken, an API rejecting a value), you apply `.is-invalid` or `.is-valid` directly to the specific field yourself, without needing `.was-validated` on the whole form:

HTML

```
<input type="text" class="form-control is-invalid" value="taken_username">
<div class="invalid-feedback">That username is already taken.</div>
```

13.3 Input Groups With Validation

HTML

```
<div class="input-group has-validation">
  <span class="input-group-text">@</span>
  <input type="text" class="form-control is-invalid" required>
  <div class="invalid-feedback">Please choose a username.</div>
</div>
```

The extra `.has-validation` class on the input-group wrapper is required whenever feedback text follows an input inside a group — it fixes a border-radius edge case so the feedback message renders below the whole group rather than being visually clipped.

13.4 Disabling the Submit Button Until Valid (JavaScript Pattern)

JAVASCRIPT

```
const forms = document.querySelectorAll('.needs-validation');
forms.forEach(form => {
  form.addEventListener('submit', event => {
    if (!form.checkValidity()) {
      event.preventDefault();
      event.stopPropagation();
    }
    form.classList.add('was-validated');
  }, false);
});
```

This is Bootstrap's officially recommended validation bootstrapping script — it uses only native browser APIs (`checkValidity()`), meaning it works even if you strip out Bootstrap's JS bundle entirely and keep only the CSS.

13.5 Advanced Pattern: A Polished Contact Form

HTML

```
<form class="row g-3 needs-validation" novalidate>
  <div class="col-12 col-md-6">
    <div class="form-floating">
      <input type="text" class="form-control" id="name" required>
      <label for="name">Full name</label>
      <div class="invalid-feedback">Please tell us your name.</div>
    </div>
  </div>
  <div class="col-12 col-md-6">
    <div class="form-floating">
      <input type="email" class="form-control" id="cemail" required>
      <label for="cemail">Email</label>
      <div class="invalid-feedback">A valid email is required.</div>
    </div>
  </div>
  <div class="col-12">
    <div class="form-floating">
      <textarea class="form-control" id="msg" style="height:120px" required></textarea>
      <label for="msg">Message</label>
      <div class="invalid-feedback">Please enter a message.</div>
    </div>
  </div>
  <div class="col-12">
    <button class="btn btn-primary btn-lg px-4" type="submit">Send message</button>
  </div>
</form>
```

TIP

The `novalidate` attribute on the `<form>` tag suppresses the browser's own default validation bubble tooltips, letting Bootstrap's styled feedback messages be the only validation UI the user sees — always pair it with the JavaScript pattern above, or validation will silently stop working altogether.

PART V · COMPONENTS

Buttons, Badges, Alerts & Cards

The four components you'll use on almost every page you ever build.

14.1 Buttons

HTML

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-outline-secondary">Outline</button>
<button class="btn btn-primary btn-lg">Large</button>
<button class="btn btn-primary btn-sm">Small</button>
<button class="btn btn-primary" disabled>Disabled</button>
<a href="#" class="btn btn-primary">A link styled as a button</a>
```

Every theme color from Chapter 8 has a matching solid (`.btn-primary`) and outline (`.btn-outline-primary`) variant — outline buttons keep the border and text colored, with a transparent background until hovered.

14.2 Button Groups and Toolbars

HTML

```
<div class="btn-group" role="group">
  <button class="btn btn-outline-primary">Day</button>
  <button class="btn btn-outline-primary">Week</button>
  <button class="btn btn-outline-primary">Month</button>
</div>
```

Wrapping buttons in `.btn-group` visually merges their borders into one segmented control by collapsing shared edges — always include `role="group"` for accessibility, since screen readers otherwise announce the buttons as unrelated.

14.3 Badges

HTML

```
<h5>Notifications <span class="badge bg-danger rounded-pill">4</span></h5>
<button class="btn btn-primary position-relative">
  Inbox
  <span class="position-absolute top-0 start-100 translate-middle badge rounded-pill
bg-danger">
    9+
  </span>
</button>
```

That second example is Bootstrap's standard "notification dot on a button" recipe — `position-absolute` pins the badge to the button, and `top-0 start-100 translate-middle` places its exact center on the button's top-right corner (a position-and-transform trick reused throughout Chapter 19's Helpers).

14.4 Alerts

HTML

```
<div class="alert alert-success alert-dismissible fade show" role="alert">
  <strong>Success!</strong> Your changes have been saved.
  <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></
button>
</div>
```

`data-bs-dismiss="alert"` is a data-attribute API call — it requires zero custom JavaScript because Bootstrap's bundled JS listens globally for clicks on any element with this attribute and handles the fade-out and removal automatically.

NOTE

Every interactive Bootstrap component (alerts, modals, dropdowns, collapses, tabs) follows this same convention: a `data-bs-*` attribute triggers behavior without you writing a line of JavaScript. This "data API" is why you can build a fully interactive Bootstrap page using only the CDN files and HTML — no custom scripting required for standard behavior.

14.5 Cards — the Most-Used Component in Bootstrap

HTML

```
<div class="card" style="width: 20rem;">
  
  <div class="card-body">
    <h5 class="card-title">Card title</h5>
    <p class="card-text">A short description supporting the title.</p>
    <a href="#" class="btn btn-primary">Go somewhere</a>
  </div>
</div>
```

A card is structurally simple: an outer `.card` (border + rounded corners + background), containing any combination of `.card-img-top`, `.card-header`, `.card-body`, and `.card-footer`. Everything inside `.card-body` uses regular typography and spacing utilities — a card has no special internal grid of its own.

14.6 Card Grids

HTML

```
<div class="row row-cols-1 row-cols-md-3 g-4">
  <div class="col"><div class="card h-100">...</div></div>
  <div class="col"><div class="card h-100">...</div></div>
  <div class="col"><div class="card h-100">...</div></div>
</div>
```

TIP

Add `.h-100` to every card in a grid row. Without it, cards with different amounts of text will render at different heights, breaking the visual rhythm of the row — `h-100` stretches each card to match the tallest column in its row (courtesy of the row's default `align-items: stretch`).

Navbar, Navs, Tabs & Dropdowns

Every navigation pattern Bootstrap ships, from a simple link list to a fully responsive collapsing navbar.

15.1 The Responsive Navbar Structure

HTML

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">Brand</a>
    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse" data-bs-target="#navMenu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto mb-2 mb-lg-0">
        <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Features</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Pricing</a></li>
      </ul>
    </div>
  </div>
</nav>
```

THE STRUCTURE

.navbar-expand-lg is the whole responsive behavior in one class: below the lg breakpoint, the link list is hidden and collapsible (toggled by the hamburger button); at lg and above, the collapse is forced open and displayed as a horizontal row automatically — no JavaScript media-query logic required, just a CSS media query flipping `display: none` to `display: flex` on the .navbar-collapse element at the chosen breakpoint.

`ms-auto` ("margin-start: auto") on the `<ul>` is what pushes the nav links to the right edge of the navbar — an ordinary Flexbox auto-margin trick, not a special navbar feature.

15.2 Navs and Tabs (Shared Base)

Navbar links, pills, and tabs all share the same base markup (`.nav` and `.nav-item` / `.nav-link`) with a different modifier class changing only the visual style.

HTML — TABS

```
<ul class="nav nav-tabs" role="tablist">
  <li class="nav-item"><button class="nav-link active" data-bs-toggle="tab"
    data-bs-target="#home">Home</button></li>
  <li class="nav-item"><button class="nav-link" data-bs-toggle="tab"
    data-bs-target="#profile">Profile</button></li>
</ul>
<div class="tab-content p-3">
  <div class="tab-pane fade show active" id="home">Home content...</div>
  <div class="tab-pane fade" id="profile">Profile content...</div>
</div>
```

HTML — PILLS

```
<ul class="nav nav-pills">
  <li class="nav-item"><a class="nav-link active" href="#">Active</a></li>
  <li class="nav-item"><a class="nav-link" href="#">Link</a></li>
</ul>
```

Swapping `.nav-tabs` for `.nav-pills` changes only the visual chrome (bordered tab shape versus a filled rounded pill) — the tab-switching JavaScript behavior triggered by `data-bs-toggle="tab"` is identical either way.

15.3 Dropdowns

HTML

```
<div class="dropdown">
  <button class="btn btn-secondary dropdown-toggle" data-bs-toggle="dropdown">
    Actions
  </button>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Edit</a></li>
    <li><a class="dropdown-item" href="#">Archive</a></li>
    <li><hr class="dropdown-divider"></li>
    <li><a class="dropdown-item text-danger" href="#">Delete</a></li>
  </ul>
</div>
```

Dropdowns rely on **Popper.js** (bundled inside `bootstrap.bundle.min.js`) to intelligently position the menu — flipping it above the button instead of below, for example, when there isn't enough space beneath it in the viewport.

15.4 Breadcrumbs

HTML

```
<nav aria-label="breadcrumb">
  <ol class="breadcrumb">
    <li class="breadcrumb-item"><a href="#">Home</a></li>
    <li class="breadcrumb-item"><a href="#">Library</a></li>
    <li class="breadcrumb-item active" aria-current="page">Data</li>
  </ol>
</nav>
```

The separator character between breadcrumb items (a slash, by default) is generated in CSS as `content` on a `::before` pseudo-element — you can restyle it entirely via the `--bs-breadcrumb-divider` CSS variable without touching the HTML.

Modals, Offcanvas, Collapse & Accordion

Four components that all share one underlying idea: hide content, then reveal it in response to a trigger.

16.1 Modals

HTML

```
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#confirmModal">
  Delete item
</button>

<div class="modal fade" id="confirmModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Confirm deletion</h5>
        <button class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">This action cannot be undone.</div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-dismiss="modal">Cancel</button>
        <button class="btn btn-danger">Delete</button>
      </div>
    </div>
  </div>
</div>
```

Structurally: the outer `.modal` is a fixed-position overlay hidden by default (`display:none`); JavaScript toggles it to visible and injects a separate `.modal-backdrop` element behind it. `.modal-dialog` centers and sizes the actual visible box; `.modal-content` is the white card holding header/body/footer.

MODIFIER	ON	EFFECT
<code>.modal-dialog-centered</code>	modal-dialog	Vertically centers the modal in the viewport
<code>.modal-dialog-scrollable</code>	modal-dialog	Makes only the modal body scroll, keeping header/footer fixed
<code>.modal-sm</code> / <code>.modal-lg</code> / <code>.modal-xl</code>	modal-dialog	Overrides the default width
<code>data-bs-backdrop="static"</code>	modal (data attr)	Clicking outside the modal no longer closes it

16.2 Offcanvas

Offcanvas is functionally a modal that slides in from an edge of the screen instead of appearing centered — the standard pattern for mobile navigation drawers and filter panels.

HTML

```
<button class="btn btn-primary" data-bs-toggle="offcanvas" data-bs-target="#filters">
  Filters
</button>

<div class="offcanvas offcanvas-start" tabindex="-1" id="filters">
  <div class="offcanvas-header">
    <h5 class="offcanvas-title">Filter results</h5>
    <button class="btn-close" data-bs-dismiss="offcanvas"></button>
  </div>
  <div class="offcanvas-body">
    ...filter checkboxes here...
  </div>
</div>
```

`.offcanvas-start` slides in from the left; swap for `-end`, `-top`, or `-bottom` for the other three edges.

16.3 Collapse

Collapse is the simplest show/hide primitive in Bootstrap, and it's the mechanism underneath the navbar toggler, accordions, and any "Read more" expander.

HTML

```
<button class="btn btn-outline-primary" data-bs-toggle="collapse" data-bs-target="#more">
  Read more
</button>
<div class="collapse" id="more">
  <p class="mt-2">This paragraph is hidden until the button is clicked,
    then animates open smoothly.</p>
</div>
```

THE STRUCTURE

Collapse animates by transitioning the `height` property from `0` to the element's measured content height (calculated by JavaScript at toggle time, since CSS alone cannot transition to/from `height: auto`), then to `height: auto` once the transition finishes so the content can still respond correctly if the window is resized.

16.4 Accordion

An accordion is several Collapse panels wired together so that opening one automatically closes the others — achieved with a shared `data-bs-parent` attribute pointing back at the outer accordion container.

HTML

```

<div class="accordion" id="faq">
  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button" data-bs-toggle="collapse" data-bs-target="#q1">
        What is Bootstrap?
      </button>
    </h2>
    <div id="q1" class="accordion-collapse collapse show" data-bs-parent="#faq">
      <div class="accordion-body">A CSS and JS toolkit for building responsive sites.</div>
    </div>
  </div>
  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button collapsed" data-bs-toggle="collapse" data-bs-target="#q2">
        Does it need jQuery?
      </button>
    </h2>
    <div id="q2" class="accordion-collapse collapse" data-bs-parent="#faq">
      <div class="accordion-body">No – Bootstrap 5 dropped the jQuery dependency.</div>
    </div>
  </div>
</div>

```

COMMON MISTAKE

Forgetting the `.collapsed` class on a closed accordion button, or `.show` on its open collapse panel, doesn't break the click functionality — but it does leave the little chevron icon pointing the wrong direction on page load, since that rotation is driven purely by the presence of the `.collapsed` CSS class, not by JavaScript checking the panel's actual state.

Carousel, Tooltips, Popovers, Toasts & Spinners

The remaining JavaScript-powered components, each solving one narrow, specific interaction.

17.1 Carousel

HTML

```
<div id="hero" class="carousel slide" data-bs-ride="carousel">
  <div class="carousel-indicators">
    <button data-bs-target="#hero" data-bs-slide-to="0" class="active"></button>
    <button data-bs-target="#hero" data-bs-slide-to="1"></button>
  </div>
  <div class="carousel-inner">
    <div class="carousel-item active"></div>
    <div class="carousel-item"></div>
  </div>
  <button class="carousel-control-prev" data-bs-target="#hero" data-bs-slide="prev">
    <span class="carousel-control-prev-icon"></span>
  </button>
  <button class="carousel-control-next" data-bs-target="#hero" data-bs-slide="next">
    <span class="carousel-control-next-icon"></span>
  </button>
</div>
```

Exactly one `.carousel-item` must carry the `.active` class at a time — this is what determines which slide is currently visible; the JavaScript's job during a transition is simply moving that class from one item to the next while sliding both horizontally with a CSS transform.

COMMON MISTAKE

Carousels autoplay images that are often the largest assets on a page. For performance and for users with vestibular motion sensitivity, always pair a carousel with `data-bs-pause="hover"` (pause on mouse hover) and respect `prefers-reduced-motion` — Bootstrap's default carousel CSS already disables the sliding transition automatically for users who have that OS-level accessibility setting turned on.

17.2 Tooltips

Unlike most components, tooltips must be manually activated with one line of JavaScript per page — Bootstrap deliberately does not auto-initialize them, since sighted users can otherwise get tooltip text read aloud unexpectedly by some assistive tech configurations if enabled globally without thought.

HTML

```
<button class="btn btn-secondary" data-bs-toggle="tooltip" data-bs-placement="top"
  title="Saved to your library">
  Save
</button>
```

JAVASCRIPT — REQUIRED ACTIVATION

```
const tooltipList = [...document.querySelectorAll('[data-bs-toggle="tooltip"]')]
  .map(el => new bootstrap.Tooltip(el));
```

17.3 Popovers

A popover is a larger tooltip that can contain a title and richer body content, triggered on click by default rather than hover.

HTML

```
<button class="btn btn-secondary" data-bs-toggle="popover" data-bs-title="Pro tip"
  data-bs-content="You can also press Ctrl+S to save.">
  ?
</button>
```

JAVASCRIPT — REQUIRED ACTIVATION

```
const popoverList = [...document.querySelectorAll('[data-bs-toggle="popover"]')]
  .map(el => new bootstrap.Popover(el));
```

17.4 Toasts

Toasts are small, auto-dismissing notification bubbles, typically stacked in a fixed corner of the screen.

HTML

```
<div class="toast-container position-fixed bottom-0 end-0 p-3">
  <div class="toast" id="saveToast" role="alert">
    <div class="toast-header">
      <strong class="me-auto">Notification</strong>
      <small>just now</small>
      <button class="btn-close" data-bs-dismiss="toast"></button>
    </div>
    <div class="toast-body">Your file has been saved.</div>
  </div>
</div>
```

JAVASCRIPT — TOASTS MUST BE SHOWN MANUALLY

```
const toast = new bootstrap.Toast(document.getElementById('saveToast'));
toast.show();
```

Toasts default to `display:none` and, unlike alerts, are never shown just by being present in the DOM — you always instantiate and call `.show()` from JavaScript, typically right after an action succeeds (a save, a form submission).

17.5 Spinners

HTML

```
<div class="spinner-border text-primary" role="status">
  <span class="visually-hidden">Loading...</span>
</div>

<button class="btn btn-primary" disabled>
  <span class="spinner-border spinner-border-sm" aria-hidden="true"></span>
  <span role="status">Loading...</span>
</button>
```

The `.visually-hidden` span (see Chapter 19) is essential — without it, a screen reader announces nothing at all while a sighted user simply sees a spinning circle, leaving assistive-technology users with no indication that content is loading.

Pagination, Progress, List Groups & Placeholders

The last four components — mostly pure CSS, no JavaScript required.

18.1 Pagination

HTML

```
<nav aria-label="Page navigation">
  <ul class="pagination">
    <li class="page-item disabled"><a class="page-link" href="#">Previous</a></li>
    <li class="page-item active"><a class="page-link" href="#">1</a></li>
    <li class="page-item"><a class="page-link" href="#">2</a></li>
    <li class="page-item"><a class="page-link" href="#">3</a></li>
    <li class="page-item"><a class="page-link" href="#">Next</a></li>
  </ul>
</nav>
```

Purely CSS-driven — `.disabled` and `.active` only change appearance and set `aria-disabled` / `aria-current` if you add them; they don't prevent a real click on their own, so a genuinely disabled link should also remove or replace its `href`.

18.2 Progress Bars

HTML

```
<div class="progress" role="progressbar" aria-valuenow="65" aria-valuemin="0" aria-
valuemax="100">
  <div class="progress-bar bg-success" style="width: 65%">65%</div>
</div>

<div class="progress">
  <div class="progress-bar progress-bar-striped progress-bar-animated" style="width:
40%"></div>
</div>
```

THE MATH

The bar's width is set directly as an inline CSS percentage matching the `aria-valuenow` attribute — Bootstrap does not calculate this for you. In a real application, this percentage is computed as $(\text{current} \div \text{max}) \times 100$ and written into both the `style="width: ...%"` and the ARIA attributes by your own JavaScript or templating logic whenever the value changes.

18.3 List Groups

HTML

```
<ul class="list-group">
  <li class="list-group-item d-flex justify-content-between align-items-center">
    Inbox
    <span class="badge bg-primary rounded-pill">12</span>
  </li>
  <li class="list-group-item active">Sent</li>
  <li class="list-group-item disabled">Archived</li>
</ul>

<div class="list-group">
  <a href="#" class="list-group-item list-group-item-action">Clickable row</a>
  <a href="#" class="list-group-item list-group-item-action active">Currently selec-
ted</a>
</div>
```

Swap the tag from `<li>` to `<a>` or `<button>` and add `.list-group-item-action` to turn a static display list into an interactive, hoverable menu — the exact structure behind most sidebar navigation menus and settings panels.

18.4 Placeholders (Loading Skeletons)

Placeholders render gray animated blocks shaped like your real content, shown while data is still loading — a modern alternative to a blank screen or a single centered spinner.

HTML

```
<div class="card" style="width: 18rem;">
  <div class="card-body">
    <h5 class="card-title placeholder-glow">
      <span class="placeholder col-6"></span>
    </h5>
    <p class="card-text placeholder-glow">
      <span class="placeholder col-7"></span>
      <span class="placeholder col-4"></span>
      <span class="placeholder col-4"></span>
      <span class="placeholder col-6"></span>
    </p>
  </div>
</div>
```

TIP

Notice that placeholders reuse the exact same `col - *` grid-width classes from Chapter 4 to control each gray bar's width — a clever reuse of an existing system rather than a brand-new width scale invented just for this component. `.placeholder-glow` adds a soft pulsing opacity animation; swap it for `.placeholder-wave` for a shimmering sweep effect instead.

PART VI · GOING PROFESSIONAL

Helper Classes

A short, high-value set of single-purpose classes that solve specific layout problems the grid and Flexbox utilities don't directly cover.

19.1 Clearfix

HTML

```
<div class="clearfix">
  
  <p>Text wraps around the floated image...</p>
</div>
```

`.clearfix` applies the classic `::after { content: ""; display: block; clear: both; }` trick to a container, forcing it to expand and contain any `.float-start` / `.float-end` children instead of collapsing to zero height around them — a legacy layout need from before Flexbox, still occasionally useful for text wrapping around an image.

19.2 Colored Links

HTML

```
<a href="#" class="link-danger link-offset-2 link-underline-opacity-25
  link-underline-opacity-100-hover">Delete account</a>
```

The `link-underline-opacity-*` family lets you fade a link's underline to near-invisible by default and bring it to full opacity only on hover — a refined, editorial-feeling link style used heavily on modern content sites.

19.3 The Ratio Helper (recap) and Position Helpers

CLASS	EFFECT
<code>.fixed-top</code> / <code>.fixed-bottom</code>	<code>position: fixed</code> , pinned to the viewport edge, stays put while scrolling
<code>.sticky-top</code>	<code>position: sticky</code> , scrolls normally until it reaches the top of the viewport, then sticks there
<code>.position-relative</code> / <code>-absolute</code> / <code>-fixed</code> / <code>-sticky</code>	sets the raw CSS <code>position</code> property
<code>.top-0</code> / <code>.bottom-0</code> / <code>.start-0</code> / <code>.end-0</code>	pins an absolutely/fixed-positioned element flush to that edge
<code>.translate-middle</code>	shifts an element back by 50% of its own width/height — used to perfectly center something <i>on</i> a coordinate rather than <i>within</i> a box

19.4 Stacks: `vstack` and `hstack`

HTML

```
<div class="vstack gap-3">
  <div>Item one</div>
  <div>Item two</div>
</div>

<div class="hstack gap-2">
  <span>Icon</span><span>Label</span>
</div>
```

`.vstack` is shorthand for `display:flex; flex-direction:column; align-self:stretch;` and `.hstack` for a horizontal, vertically-centered row — both accept the `.gap-*` spacing scale from Chapter 9 to space children evenly without needing individual margin classes on every child.

19.5 Stretched Link

HTML

```
<div class="card position-relative">
  <div class="card-body">
    <h5 class="card-title">
      <a href="/article" class="stretched-link text-decoration-none">Article title</a>
    </h5>
    <p class="card-text">A summary of the article...</p>
  </div>
</div>
```

THE STRUCTURE

`.stretched-link` uses an `::after` pseudo-element positioned `absolute` with `top:0; right:0; bottom:0; left:0;`, stretching invisibly to fill the nearest ancestor that has `position: relative` (here, the card). The result: clicking *anywhere* on the card navigates using that one small link's `href` — a common professional pattern for making an entire card "feel" clickable while keeping only one real, accessible link in the markup.

19.6 Text Truncation and Visually Hidden

HTML

```
<span class="d-inline-block text-truncate" style="max-width: 150px;">
  This long text gets cut off with an ellipsis...
</span>

<button class="btn btn-primary">
  <svg width="16" height="16"><!-- icon --></svg>
  <span class="visually-hidden">Download file</span>
</button>
```

`.text-truncate` requires a defined or constrained width and `display: inline-block` or `block` to work, because ellipsis truncation only applies to single-line block-level boxes with a fixed width to overflow against. `.visually-hidden` removes an element from sighted view (without `display:none`, which would also hide it from screen readers) — the standard way to label an icon-only button for accessibility.

Customizing Bootstrap with Sass & Dark Mode

Moving from "using Bootstrap's defaults" to "having your own design system built on Bootstrap's engine."

20.1 The Variable Override Pattern, Revisited

As introduced in Chapter 2, everything begins with overriding `!default` Sass variables before importing Bootstrap's source. The most impactful variables to change first:

CUSTOM.SCSS

```
// Brand & theme colors
$primary: #5b3df6;
$secondary: #6c757d;
$success: #12b76a;
$danger: #f04438;

// Typography
$font-family-base: 'Inter', system-ui, sans-serif;
$font-family-headings: 'Sora', sans-serif;
$headings-font-weight: 700;
$body-color: #2b2730;

// Shape & spacing
$border-radius: .625rem;
$border-radius-lg: 1rem;
$spacer: 1rem;

// Shadows
$box-shadow: 0 4px 14px rgba(0,0,0,.08);

@import "bootstrap/scss/bootstrap";
```

TIP

Changing `$border-radius` alone has an outsized effect on how "custom" a Bootstrap site feels, because it cascades into buttons, cards, inputs, modals, badges, and alerts simultaneously — one variable, a dozen components updated consistently.

20.2 Editing the Theme Color Map Directly

To add an entirely new named color (not just change an existing one), extend the `$theme-colors` map before importing:

CUSTOM.SCSS

```
@import "bootstrap/scss/functions";
@import "bootstrap/scss/variables";
@import "bootstrap/scss/variables-dark";

$theme-colors: map-merge($theme-colors, (
  "brand-pink": #ff5da2,
  "brand-navy": #1b2a4a
));

@import "bootstrap/scss/maps";
@import "bootstrap/scss/bootstrap";
```

This single addition automatically generates `.btn-brand-pink`, `.text-brand-pink`, `.bg-brand-pink`, `.badge` variants, and every other color-driven class family for your new color — for free, because those class families are all generated by looping over this one map internally.

20.3 Importing Only What You Need

Rather than importing the single umbrella `bootstrap/scss/bootstrap` file, you can import individual pieces to reduce output CSS size, which matters for performance-sensitive sites that don't use every component.

CUSTOM.SCSS

```
@import "bootstrap/scss/functions";
@import "bootstrap/scss/variables";
@import "bootstrap/scss/mixins";
@import "bootstrap/scss/root";
@import "bootstrap/scss/reboot";
@import "bootstrap/scss/type";
@import "bootstrap/scss/grid";
@import "bootstrap/scss/utilities";
@import "bootstrap/scss/utilities/api";
// Skipped: carousel, toasts, modal, offcanvas, etc — not used on this site
```

COMMON MISTAKE

Some Sass partials depend on others being imported first (functions, variables, and mixins are almost always required as a base). If you see a Sass compilation error about an undefined variable or mixin, it usually means you skipped one of these three foundational partials.

20.4 Building a Custom Dark Mode

Bootstrap 5.3's color-mode system (Chapter 8) is driven by CSS variables re-declared under a `[data-bs-theme="dark"]` selector. You can add your own custom variable overrides for dark mode the same way, in plain CSS after your compiled Bootstrap:

CSS

```
[data-bs-theme="dark"] {
  --bs-body-bg: #14121a;
  --bs-body-color: #e7e3f0;
  --bs-border-color: #322b45;
}

[data-bs-theme="dark"] .card {
  --bs-card-bg: #1e1a29;
  --bs-card-border-color: #322b45;
}
```

JAVASCRIPT — A SIMPLE THEME TOGGLE BUTTON

```
document.getElementById('themeToggle').addEventListener('click', () => {
  const html = document.documentElement;
  const current = html.getAttribute('data-bs-theme');
  html.setAttribute('data-bs-theme', current === 'dark' ? 'light' : 'dark');
  localStorage.setItem('theme', html.getAttribute('data-bs-theme'));
});
```

20.5 The Utility API, in Full

Beyond adding brand-new utilities (Chapter 9), you can also *remove* ones you never use to shrink your CSS, or *add responsive breakpoints* to a utility that doesn't have them by default:

CUSTOM.SCSS

```
$utilities: map-merge($utilities, (  
  "cursor": null,    // removes an unused utility entirely  
  "border-radius": map-merge(  
    map-get($utilities, "border-radius"),  
    ( responsive: true )           // adds .rounded-md-3, .rounded-lg-circle, etc.  
  )  
));
```

Design Principles: Making Bootstrap Look Custom

Technical mastery of every class in this book still won't produce a beautiful page without design judgment. This chapter is the missing piece.

21.1 Why Default Bootstrap Sites Look "Cheap"

It is rarely the components themselves — it's the accumulation of default choices left untouched: the default blue, the default 0.375rem radius on every corner, the default system font, and — most commonly — insufficient whitespace, because Bootstrap's own spacing defaults are deliberately modest and neutral, expecting *you* to add generous breathing room for a premium feel.

21.2 Whitespace Is a Design Decision, Not a Gap You Forgot

TIP

Professional marketing sites typically use far more vertical padding around sections than beginners expect — `py-5` (48px) is often a bare *minimum* for a section, with many premium sites using custom values of 6–10rem on desktop. Generous whitespace is one of the strongest, cheapest visual signals of quality design.

HTML

```
<section class="py-5 py-lg-6"> <!-- py-lg-6 requires adding a "6" step to $spacers in
Sass -->
  ...
</section>
```

21.3 Establish a Strict Type Scale — Don't Freelance Font Sizes

Pick 4–5 font sizes total for the entire site (for example: a display size for hero headlines, an h2 size for section titles, a body size, and a small/caption size) and never introduce a size outside that scale. Bootstrap's `.fs-1` – `.fs-6` and heading tags already give you a built-in scale — the discipline is refusing to add a one-off `style="font-size:19px"` anywhere.

21.4 Shadows and Depth, Used Sparingly

HTML

```
<div class="card shadow-sm">Subtle elevation for a resting card</div>
<div class="card shadow">Standard elevation, e.g. a dropdown or popover</div>
<div class="card shadow-lg">Strong elevation, e.g. a modal or hovering hero card</div>
```

A common professional micro-interaction: apply a slightly deeper shadow and a small upward transform on hover, using a custom CSS transition (not a Bootstrap utility, since Bootstrap has no built-in hover-shadow utility):

CSS

```
.card-hover {
  transition: transform .2s ease, box-shadow .2s ease;
}
.card-hover:hover {
  transform: translateY(-4px);
  box-shadow: 0 12px 24px rgba(0,0,0,.12);
}
```

21.5 Break the Grid Occasionally, on Purpose

Perfectly even, symmetrical `col-4 col-4 col-4` rows everywhere can look generic. Professional layouts often use deliberately asymmetric splits — a `col-lg-7` / `col-lg-5` pairing for a hero image and text, for instance — which still uses the same 12-column math from Chapter 4, just avoiding the most predictable even divisions.

21.6 Reduce Border Usage; Prefer Background and Spacing to Separate Content

Beginners often reach for `.border` to separate every card or section. Experienced designers more often separate content using whitespace and subtle background-color shifts (`.bg-light` alternating with white sections) instead, reserving visible borders for places that genuinely need a hard edge, like form inputs.

21.7 One Accent Color, Used Consistently

Pick one accent color (usually your `$primary`) and use it *only* for the single most important action per screen — the one button or link you most want clicked. Every other button on the page should be an outline or secondary/neutral style. A page with five equally-loud primary-colored buttons has no visual hierarchy at all.

21.8 Icons: Pairing Bootstrap With Bootstrap Icons

Bootstrap Icons is a separate, free SVG icon set built by the same team, sized to align naturally with Bootstrap's type scale and spacing.

HTML

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/
bootstrap-icons.css">

<button class="btn btn-primary">
  <i class="bi bi-cloud-arrow-up"></i> Upload
</button>
```

TIP

A small but noticeable detail in polished interfaces: icons and their adjacent text should share the same optical color and roughly the same visual weight. Avoid mixing a heavy, filled icon glyph next to a very light-weight font — the mismatch subtly reads as unpolished even when the individual pieces are fine.

Performance, Accessibility & Best Practices

The engineering discipline that separates a portfolio project from a production-ready website.

22.1 Only Ship the CSS You Use

The full compiled Bootstrap stylesheet is large because it includes every component. Two practical strategies reduce this: import only the Sass partials you actually use (Chapter 20.3), or run a build-time "unused CSS" removal tool such as PurgeCSS against your final HTML, which scans your markup for class names actually present and strips everything else from the shipped stylesheet.

22.2 Load JavaScript Only If You Need It

TIP

If your page has no dropdowns, modals, tooltips, popovers, carousels, or offcanvas — nothing that requires `bootstrap.bundle.min.js` — don't load it. A pure-content or pure-layout page only needs Bootstrap's CSS file, saving a real, measurable amount of parse and execution time, especially on slower mobile devices.

22.3 Defer Non-Critical JavaScript

HTML

```
<script src="bootstrap.bundle.min.js" defer></script>
```

The `defer` attribute lets the browser continue parsing and rendering the HTML document while the script downloads in the background, only executing it once parsing is complete — this avoids JavaScript blocking the visible rendering of your page.

22.4 Accessible Markup Checklist

- Every `<img>` needs a meaningful `alt` attribute (or `alt=""` if the image is purely decorative).

- Icon-only buttons need a `.visually-hidden` text label (Chapter 19.6) — an icon alone conveys nothing to a screen reader.
- Use real semantic elements (`<nav>` , `<button>` , `<header>`) rather than generic `<div>` s wherever the semantic tag exists — Bootstrap's classes work identically regardless of the tag they're on, so there's no reason to sacrifice semantics for styling.
- Never remove focus outlines (`outline: none`) without providing a clearly visible custom focus style in its place — keyboard-only users rely on this to know where they are on the page.
- Check color contrast for custom colors, especially for text on tinted or colored backgrounds — Bootstrap's own defaults are chosen to meet WCAG AA contrast, but a custom `$primary` may not automatically.

22.5 Test at Every Breakpoint, Not Just Desktop and One Phone Size

A page that looks correct at exactly 375px and 1440px can still break at in-between widths, particularly right around a breakpoint threshold where columns wrap or a navbar collapses. Resize a real browser window slowly across the full range, or use browser dev tools' responsive mode, rather than only checking two fixed device presets.

22.6 Avoid Overriding Bootstrap With `!important`

COMMON MISTAKE

Reaching for `!important` to force an override is almost always a sign that a Sass variable should have been changed instead (Chapter 20), or that a more specific selector should be used. Scattering `!important` through a stylesheet makes every future style change progressively harder to reason about, since specificity no longer determines which rule wins.

22.7 Keep a Consistent Class Order in Your Markup

While CSS class order never affects rendering, adopting a consistent personal or team convention — for example: layout classes, then spacing, then color, then typography — makes long class lists on complex elements dramatically easier to scan and edit later.

HTML — A READABLE, CONSISTENTLY ORDERED CLASS LIST

```
<div class="d-flex align-items-center p-3 mb-2 bg-light border rounded fw-bold text-primary">
```

22.8 Validate Your HTML

Because Bootstrap relies on specific parent-child structures (a `.dropdown-menu` as a sibling of `.dropdown-toggle`, a `.collapse` matching its trigger's `data-bs-target`), most "broken component" bugs in practice are actually invalid or mismatched HTML rather than a CSS problem — running your markup through the W3C validator catches many of these before they cost you a debugging session.

Capstone Project: A Complete Landing Page

Every concept from the previous 22 chapters, combined into one real, professional-looking page, built section by section.

23.1 Planning the Page

We'll build a SaaS-style marketing landing page with seven sections: a navbar, a hero, a feature grid, a stats band, a testimonial, a pricing table, and a footer. Before writing markup, we make three design decisions per Chapter 21: one accent color (a custom indigo, overriding `$primary`), generous section padding (`py-5 py-lg-6`), and a restrained type scale using only display, h2, body, and small sizes.

23.2 The Navbar

HTML

```
<nav class="navbar navbar-expand-lg bg-white shadow-sm fixed-top">
  <div class="container">
    <a class="navbar-brand fw-bold text-primary" href="#">Northwind</a>
    <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#nav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="nav">
      <ul class="navbar-nav ms-auto align-items-lg-center gap-lg-4">
        <li class="nav-item"><a class="nav-link" href="#features">Features</a></li>
        <li class="nav-item"><a class="nav-link" href="#pricing">Pricing</a></li>
        <li class="nav-item"><a class="btn btn-primary rounded-pill px-3" href="#">Get
started</a></li>
      </ul>
    </div>
  </div>
</nav>
```

`.fixed-top` (Chapter 19) keeps the navbar pinned while scrolling; `.shadow-sm` gives it just enough separation from content beneath it. Because the navbar is fixed, remember to add top padding to the `<body>` equal to the navbar's rendered height, or the hero section will render underneath it.

23.3 The Hero — Asymmetric Grid on Purpose

HTML

```
<header class="pt-5 mt-5 pb-5">
  <div class="container py-5">
    <div class="row align-items-center g-5">
      <div class="col-lg-6">
        <span class="badge text-bg-primary-subtle text-primary-emphasis rounded-pill
mb-3 px-3 py-2">
          New – v2.0 is here
        </span>
        <h1 class="display-4 fw-bold mb-3">Ship your product pages in days, not
weeks.</h1>
        <p class="lead text-secondary mb-4">Northwind gives your team a shared compon-
ent
          library so every landing page looks consistent and professional.</p>
        <div class="d-flex gap-3">
          <a href="#" class="btn btn-primary btn-lg px-4 rounded-pill">Start free tri-
al</a>
          <a href="#" class="btn btn-outline-secondary btn-lg px-4 rounded-pill">Watch
demo</a>
        </div>
      </div>
      <div class="col-lg-6">
        
      </div>
    </div>
  </div>
</header>
```

Per Chapter 21.5, the hero deliberately uses a 6/6 split rather than an uneven ratio here because both sides (headline block and image) carry roughly equal visual weight — but note the badge, lead paragraph, and two-button group are themselves a small internal hierarchy: one filled primary action, one outline secondary action, never two filled buttons competing for attention.

23.4 Feature Grid

HTML

```
<section id="features" class="py-5 py-lg-6 bg-light">
  <div class="container">
    <div class="text-center mx-auto mb-5" style="max-width:600px;">
      <h2 class="fw-bold">Everything you need, nothing you don't</h2>
      <p class="text-secondary">Three focused tools instead of forty half-finished
ones.</p>
    </div>
    <div class="row row-cols-1 row-cols-md-3 g-4">
      <div class="col">
        <div class="card h-100 border-0 shadow-sm p-3">
          <div class="card-body">
            <div class="fs-2 text-primary mb-3"><i class="bi bi-lightning-charge-
fill"></i></div>
            <h5 class="card-title fw-bold">Fast by default</h5>
            <p class="card-text text-secondary">Pages ship pre-optimized, no extra con-
figuration.</p>
          </div>
        </div>
      </div>
      <!-- repeat .col twice more with different icon/title/text -->
    </div>
  </div>
</section>
```

Consistent with Chapter 21.6, the cards use `border-0 shadow-sm` rather than a visible border — separation comes from the soft shadow and the section's light gray background, not a hard rule.

23.5 Stats Band

HTML

```
<section class="py-5 bg-dark text-white">
  <div class="container">
    <div class="row text-center g-4">
      <div class="col-6 col-md-3">
        <div class="display-6 fw-bold">12k+</div>
        <div class="text-white-50 small text-uppercase">Teams</div>
      </div>
      <div class="col-6 col-md-3">
        <div class="display-6 fw-bold">99.98%</div>
        <div class="text-white-50 small text-uppercase">Uptime</div>
      </div>
      <!-- two more stat columns -->
    </div>
  </div>
</section>
```

This is `row-cols` math without the class: `col-6` gives two stats per row on phones, `col-md-3` gives four per row from tablet up — the exact pattern from Chapter 4.7, written explicitly here for full control over which two stats pair up on the smallest screens.

23.6 Pricing Table

HTML

```
<section id="pricing" class="py-5 py-lg-6">
  <div class="container">
    <div class="row row-cols-1 row-cols-md-3 g-4 justify-content-center">
      <div class="col">
        <div class="card h-100 border-0 shadow-sm text-center p-4">
          <h5 class="fw-bold">Starter</h5>
          <div class="display-6 fw-bold my-3">$9<small class="fs-6 text-secondary">/
mo</small></div>
          <ul class="list-unstyled text-secondary mb-4">
            <li class="mb-2">5 projects</li><li class="mb-2">Basic support</li>
          </ul>
          <a href="#" class="btn btn-outline-primary rounded-pill mt-auto">Choose
plan</a>
        </div>
      </div>
      <div class="col">
        <div class="card h-100 border-primary border-2 shadow text-center p-4 position-
relative">
          <span class="badge bg-primary position-absolute top-0 start-50 translate-
middle rounded-pill px-3">
            Most popular
          </span>
          <h5 class="fw-bold">Pro</h5>
          <div class="display-6 fw-bold my-3">$29<small class="fs-6 text-secondary">/
mo</small></div>
          <ul class="list-unstyled text-secondary mb-4">
            <li class="mb-2">Unlimited projects</li><li class="mb-2">Priority support</
li>
          </ul>
          <a href="#" class="btn btn-primary rounded-pill mt-auto">Choose plan</a>
        </div>
      </div>
      <!-- third "Enterprise" column follows the same pattern as Starter -->
    </div>
  </div>
</section>
```

The "Most popular" badge reuses the exact `position-absolute top-0 start-50 translate-middle` recipe from Chapter 14.3 — the same three utility classes, just centered horizontally at the top edge instead of a corner, proving how a small set of position helpers recombines to solve many different placement problems.

23.7 Footer

HTML

```
<footer class="bg-dark text-white-50 py-5">
  <div class="container">
    <div class="row g-4">
      <div class="col-md-4">
        <h5 class="text-white fw-bold">Northwind</h5>
        <p class="small">Building better landing pages since 2024.</p>
      </div>
      <div class="col-6 col-md-2">
        <h6 class="text-white">Product</h6>
        <ul class="list-unstyled small vstack gap-2">
          <li><a href="#" class="link-light link-underline-opacity-25">Features</a></li>
          <li><a href="#" class="link-light link-underline-opacity-25">Pricing</a></li>
        </ul>
      </div>
      <!-- two more footer columns follow the same pattern -->
    </div>
    <hr class="border-secondary my-4">
    <div class="d-flex flex-column flex-md-row justify-content-between small">
      <span>© 2026 Northwind, Inc.</span>
      <span>Privacy · Terms</span>
    </div>
  </div>
</footer>
```

23.8 What This Page Demonstrates

Every section above reuses the same small vocabulary: the 12-column grid (Ch. 4–6), the spacing scale (Ch. 9), Flexbox utilities (Ch. 10), cards and badges (Ch. 14), the navbar (Ch. 15), and the position helpers (Ch. 19) — recombined seven different ways. This is the core insight of the entire book: Bootstrap mastery isn't memorizing more classes, it's getting fluent enough with a small, well-understood set that you can assemble them into new layouts confidently and quickly.

Cheat Sheet & Class Reference

A compressed, at-a-glance reference for the classes you'll reach for in almost every project.

24.1 Breakpoints

INFIX	MIN WIDTH
(none)	0px
sm	576px
md	768px
lg	992px
xl	1200px
xxl	1400px

24.2 Grid

NEED	CLASS PATTERN
Column width	<code>col-{bp}-{1-12}</code>
Equal-width auto column	<code>col</code>
Offset	<code>offset-{bp}-{1-11}</code>
Order	<code>order-{0-5}</code> , <code>order-first</code> , <code>order-last</code>
Columns per row (on the row)	<code>row-cols-{bp}-{1-6}</code>
Gutter size	<code>g-{0-5}</code> , <code>gx-*</code> , <code>gy-*</code>
Container	<code>container</code> , <code>container-fluid</code> , <code>container-{bp}</code>

24.3 Spacing (m/p × t/b/s/e/x/y × 0–5/auto)

STEP	REM	PX
0	0	0
1	.25	4
2	.5	8
3	1	16
4	1.5	24
5	3	48

24.4 Flexbox

PURPOSE	CLASSES
Enable flex	<code>d-flex</code> , <code>d-inline-flex</code>
Direction	<code>flex-row</code> , <code>flex-column</code> (+ <code>-reverse</code>)
Wrap	<code>flex-wrap</code> , <code>flex-nowrap</code>
Main axis	<code>justify-content-{start end center between around evenly}</code>
Cross axis (all)	<code>align-items-{start end center baseline stretch}</code>
Cross axis (one item)	<code>align-self-*</code>
Grow / shrink	<code>flex-grow-{0 1}</code> , <code>flex-shrink-{0 1}</code> , <code>flex-fill</code>
Gap	<code>gap-{0-5}</code>

24.5 Display & Visibility

CLASS	EFFECT
<code>d-none</code> / <code>d-{bp}-none</code>	hide at this breakpoint and up
<code>d-{bp}-block</code>	show as block at this breakpoint and up
<code>visually-hidden</code>	hide visually, keep for screen readers

24.6 Color & Background

`text-{color}`, `bg-{color}`, `text-bg-{color}`, `border-{color}`, `btn-{color}`, `btn-outline-{color}`, `badge bg-{color}`, `alert-{color}` — where `{color}` is one of `primary`, `secondary`, `success`, `danger`, `warning`, `info`, `light`, `dark`.

24.7 Typography

NEED	CLASS
Big headline	<code>display-1</code> to <code>display-6</code>
Intro paragraph	<code>lead</code>
Weight	<code>fw-bold</code> , <code>fw-normal</code> , <code>fw-light</code>
Style	<code>fst-italic</code>
Alignment	<code>text-start</code> , <code>text-center</code> , <code>text-end</code>
Muted	<code>text-muted</code> , <code>text-secondary</code>
Truncate	<code>text-truncate</code>

24.8 Border & Shape

CLASS	EFFECT
<code>border</code> / <code>border-0</code>	add / remove all borders
<code>border-top/end/bottom/start</code>	one side only
<code>rounded</code> , <code>rounded-{0-3}</code> , <code>circle</code> , <code>pill</code>	corner radius presets
<code>shadow-sm</code> / <code>shadow</code> / <code>shadow-lg</code> / <code>shadow-none</code>	elevation

24.9 Position

CLASS	EFFECT
<code>position-static/relative/absolute/fixed/sticky</code>	sets CSS position
<code>top-0/50/100</code> , <code>start-0/50/100</code> , <code>end-*</code> , <code>bottom-*</code>	edge offsets (percentages)
<code>translate-middle</code> , <code>translate-middle-x/y</code>	shift back by 50% of own size
<code>fixed-top/bottom</code> , <code>sticky-top</code>	common navbar/toolbar positioning

24.10 Sizing

CLASS	EFFECT
<code>w-25/50/75/100/auto</code>	width, % of parent
<code>h-25/50/75/100/auto</code>	height, % of parent
<code>mw-100</code>	max-width: 100%
<code>vw-100</code> / <code>vh-100</code>	100% of viewport width/height
<code>min-vh-100</code>	minimum full-viewport height

24.11 Data-Attribute API Quick Reference

COMPONENT	TRIGGER ATTRIBUTE
Modal	<code>data-bs-toggle="modal" data-bs-target="#id"</code>
Collapse / Accordion	<code>data-bs-toggle="collapse" data-bs-target="#id"</code>
Dropdown	<code>data-bs-toggle="dropdown"</code>
Tab	<code>data-bs-toggle="tab" data-bs-target="#id"</code>
Offcanvas	<code>data-bs-toggle="offcanvas" data-bs-target="#id"</code>
Dismiss (alert/modal/toast)	<code>data-bs-dismiss="alert modal toast"</code>
Tooltip / Popover	<code>data-bs-toggle="tooltip popover"</code> + JS init required

You've now covered every major system in Bootstrap 5 — the grid's arithmetic, the utility engine's naming logic, every component's markup and behavior, Sass-level customization, and the design judgment to make it all look intentional rather than default. The fastest way to cement this is to rebuild the Chapter 23 capstone page from memory, then design one of your own from a blank file.